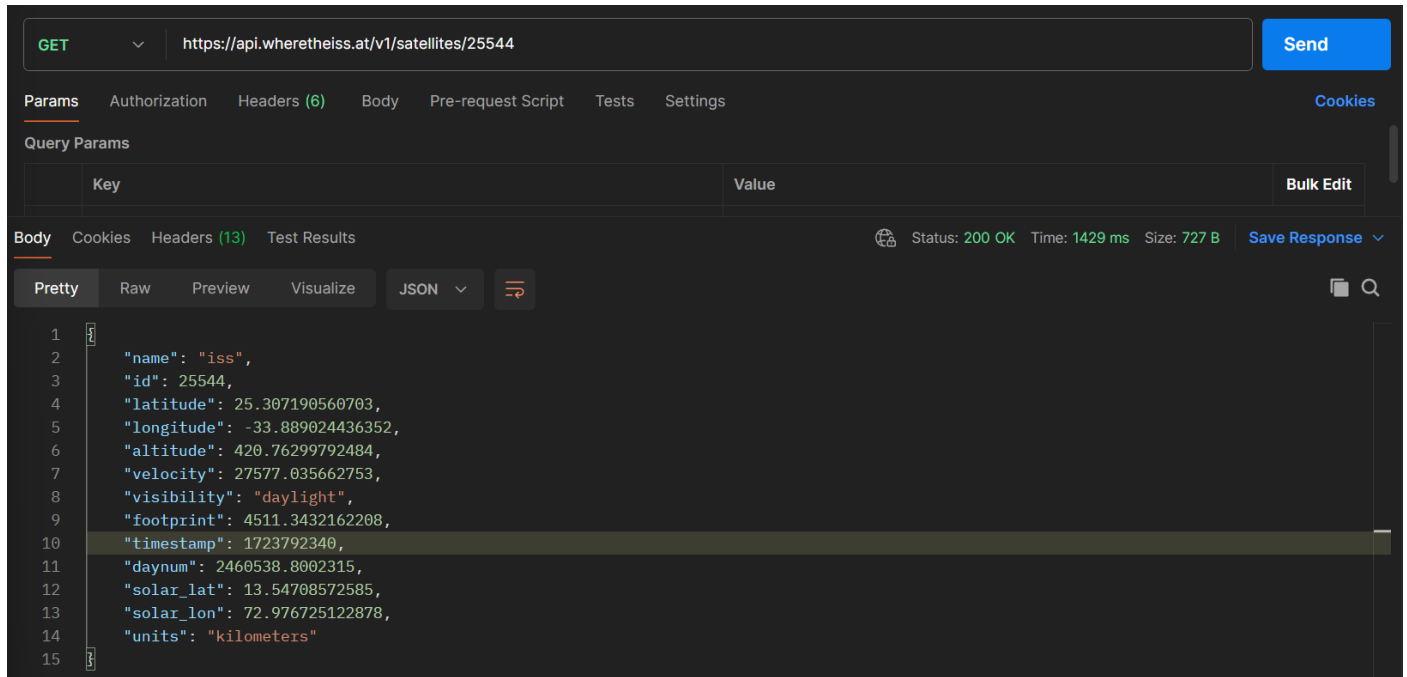


224. Introduction to APIs

- API: Application Programming Interface.
- A set of rules or protocols that tells applications how to interact with each other.
- Type of APIs by architectural structure:
 - GraphQL
 - REST:API
 - SOAP
 - gRPC
- REST:API uses HTTP protocols (GET, POST etc.).



225. Structuring API Requests

- API Endpoint:
 - BaseURL/Endpoint
 - E.g. <https://bored-api-appbrewery.com/random>; where random is an endpoint.
- Query Parameters:
 - BaseURL/Endpoint?query1=value1&query2=value2
 - E.g. <https://bored-api-appbrewery.com/filter?type=education>; where `filter?type=education` is a query parameter.
- Path parameters:
 - BaseURL/Endpoint/{path-parameter}
 - E.g. <https://bored-api-appbrewery.com/activity/3943506>; where 394350 is path parameter.

226. What is JSON?

- JSON: JavaScript object notation.
- It's a way to format data that can be sent over the internet in a readable format.
- Unlike JS object, the keys are strings.
- JS Object -> JSON

- o `const jsonData = JSON.stringify(data);`
- JSON -> JS Object
 - o `const data = JSON.parse(jsonDATA);`

227. Making Server-Side API Requests with Axios

- Axios is a popular JavaScript library used for making HTTP requests from the browser or Node.js. It is a promise-based HTTP client that works on both client and server-side, allowing developers to perform various network requests, such as GET, POST, PUT, DELETE, etc.
- Example usage:

```
import express from "express";
import bodyParser from "body-parser";
import axios from "axios";
const APP = express();
const PORT = 3000;
APP.use(express.static("public"));
APP.use(bodyParser.urlencoded({ extended: true }));

APP.get("/", async (req, res) => {
  try {
    const response = await axios.get("https://bored-api.appbrewery.com/random");
    const result = response.data;
    console.log(result);
    res.render("index.ejs", { data: result });
  } catch (error) {
    console.error("Failed to make request:", error.message);
    res.render("index.ejs", { error: error.message });
  }
});

APP.post("/", async (req, res) => {
  try {
    console.log(req.body);
    const type = req.body.type;
    const participants = req.body.participants;
    const response = await axios.get (
      `https://bored-api.appbrewery.com/filter?type=${type}&participants=${participants}`
    );
    const result = response.data;
    console.log(result);
```

```

        res.render("index.ejs", {
            data: result[Math.floor(Math.random() * result.length)]
        });
    } catch (error) {
        console.error("Failed to make request:", error.message);
        res.render("index.ejs", {
            error: "No activities that match your criteria.",
        });
    }
});

APP.listen(PORT, () => {
    console.log(`Server running on PORT: ${PORT}`);
});

```

228. API Authentication and Authorisation

- API authentication is the process of verifying the identity of an application or user attempting to access an API.
- This ensures that only authorized users or applications can interact with the API, protecting sensitive data and functionalities from unauthorized access.
- There are several methods of API authentication, each with its own use cases and security implications. Here are some common methods:
 - No authentication (rate limiting):
 - Uses requests/timeUnit(s).
 - E.g. 100 requests/15 minutes

```
const result = await axios.get(`${API_URL}/random`);
```

- Basic authentication
 - Uses a username and password.
 - We provide these credentials while getting response from API.
 - Uses Base64 encoding.
 - {


```

                        username:
                        password:
                    
```

```

const result = await axios.get(`${API_URL}/all?page=2`, {
    auth: {
        username: yourUsername,
        password: yourPassword
    }
});

```

- API key authorisation:

- **Authentication** is the process of verifying the identity of a user or application, ensuring they are who they claim to be.
Authorization determines what an authenticated user or application is allowed to do, granting or denying permissions to access resources or perform actions.
- API key authorization is a method of securing API access by requiring a unique key to accompany each API request. This key acts as a token of authentication and authorization, verifying that the request originates from a legitimate source.\
- `baseUrl/endpoint&apiKey=xxxx`

```
const result = await axios.get(`${API_URL}/filter`, {
  params: {
    score: 5,
    apiKey: yourAPIKey
  }
});
```

- Token based authentication
 - User logs in with username and password and then a token is generated.
 - The token is used to interact with API.
 - OAuth is used for token-based authorisation.

```
const result = await axios.get(`${API_URL}/secrets/42`, {
  headers: {
    Authorization: `Bearer ${yourBearerToken}`
  }
});
```

229. REST APIs

- There is an alternative way of calling axios where we chain `.then`, `.catch` and even `.finally`; `.then` gets triggered only when a promise is fulfilled. This is an old way and not in use now.
- ``async`` allows the function to operate asynchronously, meaning it can perform non-blocking operations. Inside an async function, you can use the `await` keyword to wait for asynchronous operations to complete.
- ``await`` makes the code wait for the promise to resolve and returns the resolved value. If the promise is rejected, it throws an error which can be caught using `try...catch`.
- HTTP protocols supported by axios (REST architectural design):
 - `axios.get(url[, config])`: Sends a GET request to the specified URL to retrieve data.

```
const result = await axios.get(`${API_URL}/secrets/${searchId}`,
config);
```

- **axios.post(url, data[, config]):** Sends a POST request to the specified URL with the provided data to create a new resource.

```
const result = await axios.post(`${API_URL}/secrets`, req.body, config)
```

- **axios.put(url, data[, config]):** Sends a PUT request to the specified URL with the provided data to completely update an existing resource.

```
const result = await axios.put(`${API_URL}/secrets/${searchId}`, req.body, config)
```

- **axios.patch(url, data[, config]):** Sends a PATCH request to the specified URL with the provided data to partially update an existing resource.

```
const result = await axios.patch(`${API_URL}/secrets/${searchId}`, req.body, config)
```

- **axios.delete(url[, config]):** Sends a DELETE request to the specified URL to remove a resource.

```
const result = await axios.delete(`${API_URL}/secrets/${searchId}`, config)
```

- Request method aliases:

- **axios.request(config)**
- **axios.get(url[, config])**
- **axios.delete(url[, config])**
- **axios.head(url[, config])**
- **axios.options(url[, config])**
- **axios.post(url[, data[, config]])**
- **axios.put(url[, data[, config]])**
- **axios.patch(url[, data[, config]])**
- **axios.postForm(url[, data[, config]])**
- **axios.putForm(url[, data[, config]])**
- **axios.patchForm(url[, data[, config]])**
- [] => Optional parameters.